Active Effects System Architecture

Complete File Map & Responsibility Guide

Purpose: Understanding which file is responsible for what in the Active Effects calculation chain

Last Updated: 2024-02-14

📊 TABLE OF CONTENTS

🎯 SYSTEM OVERVIEW

What are "Active Effects"?

Active Effects represent the **current and projected blood glucose impact** from:

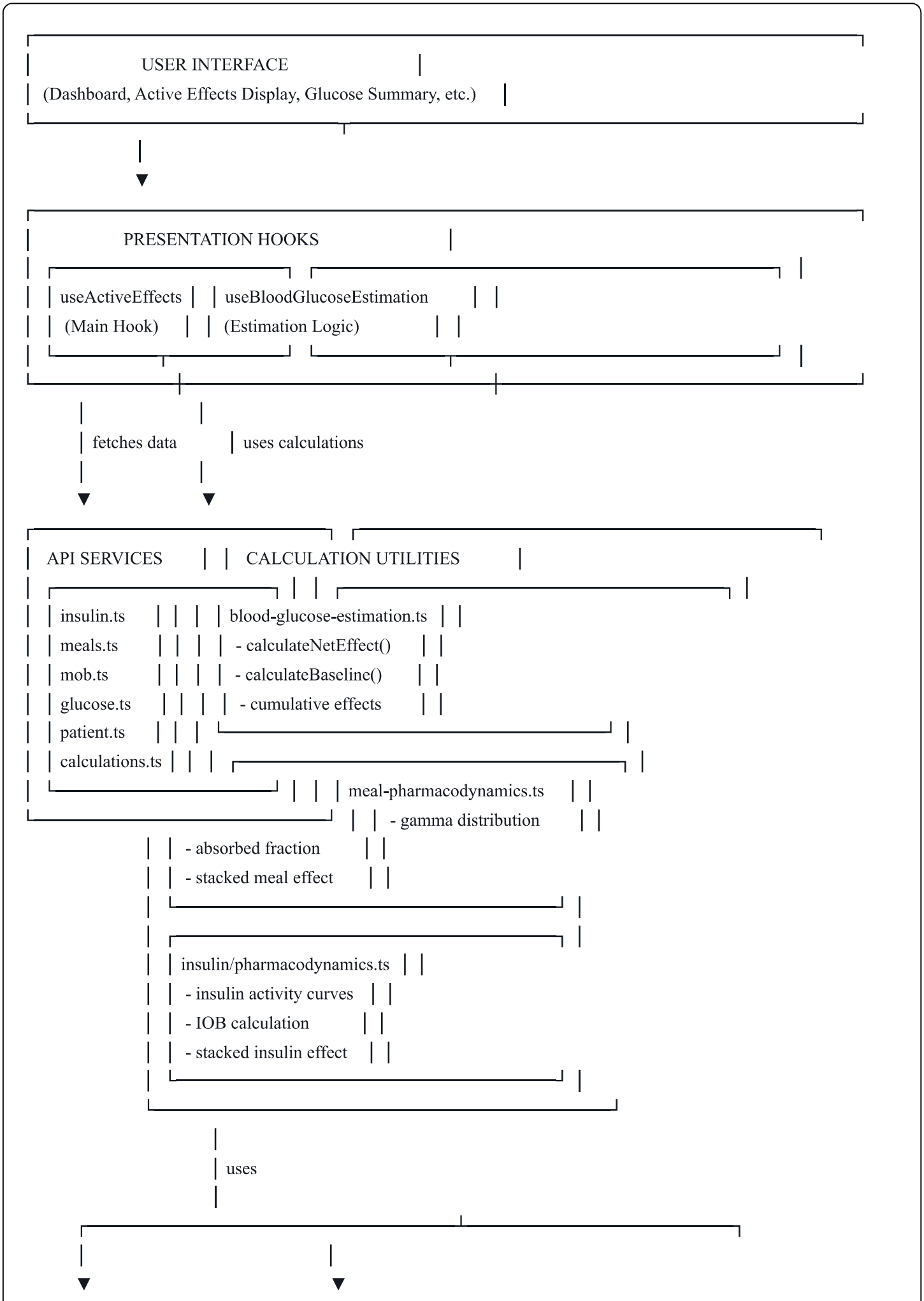
- **IOB (Insulin On Board) -** Insulin still active in the body
- **MOB (Meal On Board) -** Carbohydrates still being absorbed
- **Net Effect -** Combined impact (meals pushing up, insulin pushing down)
- **Baseline -** Stable metabolic state
- **Projected BG -** Where blood glucose is heading

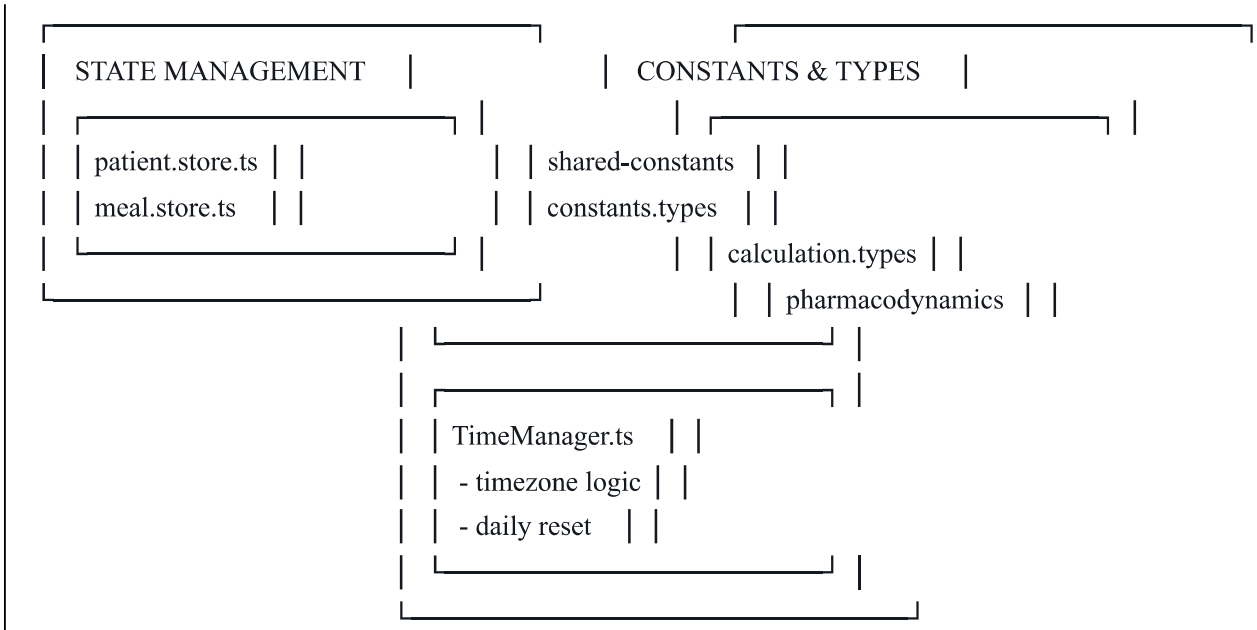
The Complete Calculation:

```
Estimated BG = Baseline + Cumulative Net Effect
             = Baseline + (Cumulative Meal - Cumulative Insulin)

Projected BG = Baseline + Cumulative + (Pending MOB Rise - Pending IOB Drop)
```

DATA FLOW ARCHITECTURE





📁 FILE DIRECTORY MAP

```
mobile/
├── hooks/                          # Presentation Layer
│   ├── useActiveEffects.ts         ⭐ MAIN HOOK - Orchestrates everything
│   ├── useBloodGlucoseEstimation.ts ⭐ BG Estimation Logic
│   └── usePatientConstants.ts      📋 Constants Provider
│
├── utils/
│   ├── glucose/                    # Meal & BG Calculations
│   │   ├── blood-glucose-estimation.ts ⭐⭐⭐ CORE ENGINE
│   │   ├── meal-pharmacodynamics.ts    ⭐⭐ Meal Absorption
│   │   ├── meal-impact.ts              📊 Meal BG Impact
│   │   ├── meal-impact-curves.ts       📈 Absorption Curves
│   │   ├── carb-equivalents.ts         🔢 Carb Math
│   │   └── baseline.ts                 📍 Re-export wrapper
│   │
│   ├── insulin/                    # Insulin Calculations
│   │   ├── pharmacodynamics.ts         ⭐⭐ Insulin Activity
│   │   └── pharmacokinetics.ts         💉 S3 Guidelines
│   │
│   ├── calculations/               # High-Level Wrappers
│   │   ├── net-effect.ts               🎯 Net Effect Wrapper
│   │   ├── cumulative-effects.ts       📊 Cumulative Wrapper
│   │   └── baseline.ts                 📍 Baseline Wrapper
│   │
│   ├── time/                       # Time Management
│   │   └── TimeManager.ts              ⏰ Time Utilities
│   │
```

```
│   └── validation/              # Input Validation
│       └── input-validators.ts  ✅ Data Validation
│
├── services/api/                # Backend Communication
│   ├── insulin.ts               💉 IOB API
│   ├── meals.ts                 🍽️ Meals API
│   ├── mob.ts                   📊 MOB API
│   ├── glucose.ts               🩸 Glucose API
│   ├── patient.ts               👤 Patient Constants API
│   ├── calculations.ts          🧮 Calculations API
│   ├── client.ts                🌐 Axios Client
│   └── endpoints.ts             🔗 URL Mapping
│
├── store/                       # State Management
│   ├── patient.store.ts         👤 Patient State
│   └── meal.store.ts            🍽️ Meal State
│
├── constants/                   # Constants & Configs
│   ├── shared-constants.ts      📋 Main Constants
│   └── index.ts                 📦 Barrel Export
│
└── types/                       # Type Definitions
    ├── calculation.types.ts     🧮 Calculation Types
    ├── pharmacodynamics.types.ts 💊 PD/PK Types
    ├── constants.types.ts       📋 Constants Types
    ├── meal.types.ts            🍽️ Meal Types
    ├── insulin.types.ts         💉 Insulin Types
    ├── glucose.types.ts         🩸 Glucose Types
    └── api.ts                   🌐 API Response Types
```

Legend:

- ⭐⭐⭐ = Critical core file (highest priority for debugging)
- ⭐⭐ = Important calculation file
- ⭐ = Primary integration point
- 📊 = Data structure/calculation
- 🔢 = Math utilities
- 📋 = Configuration
- 🌐 = Network/API
- 👤 = State management

📄 DETAILED FILE BREAKDOWN

TIER 1: CRITICAL CORE FILES (Start Here for Debugging)

1. `hooks/useActiveEffects.ts` ⭐

Role: Main orchestrator hook that combines all active effects

Responsibilities:

- Fetches meals, insulin doses, glucose readings from API
- Calls calculation utilities to compute IOB, MOB, baseline, net effect
- Manages loading states and error handling
- Provides unified interface to UI components
- Refreshes data on demand

Key Functions:

```typescript
export function useActiveEffects() {
  // Fetches all data
  // Calculates active effects
  // Returns: { iob, mob, netEffect, baseline, ... }
}
```

Used By:

- Dashboard components
- Active Effects Display
- Glucose Summary
- Any component showing current BG status

Calls:

- `getDoses()` from `services/api/insulin.ts`
- `getMeals()` from `services/api/meals.ts`
- `getReadings()` from `services/api/glucose.ts`
- `calculateNetEffect()` from `utils/glucose/blood-glucose-estimation.ts`
- `calculateStableBaselineFromReading()` from `utils/glucose/blood-glucose-estimation.ts`

Debug Here If:

- Data not showing in UI
- Loading states stuck
- Missing IOB/MOB values

- API errors

---

2. `utils/glucose/blood-glucose-estimation.ts` ⭐⭐⭐

Role: THE CORE CALCULATION ENGINE - All BG calculations happen here

Responsibilities:

- **Daily Reset Logic** - Determines when cumulative effects reset
- **Baseline Calculation** - Derives stable baseline from readings
- **Cumulative Effects** - Calculates absorbed carbs and insulin
- **Net Effect** - Combines everything into estimated BG
- **Safety Status** - Determines if BG is safe/risky
- **Trend Detection** - Rising/falling/stable

Key Functions:

```typescript
// Daily reset
getDailyResetTime(date, resetHour, timezoneOffset): Date
getLastResetTime(currentTime, resetHour, timezoneOffset): Date

// Cumulative effects (PAST→PRESENT: what's already absorbed)
calculateMealCumulativeEffect(meal, currentTime, constants, ...): number
calculateInsulinCumulativeEffect(dose, currentTime, constants, ...): number
calculateTotalCumulativeEffects(meals, doses, ...): CumulativeEffectsResult

// Baseline (stable metabolic state)
calculateStableBaselineFromReading(reading, meals, doses, ...): BaselineResult

// Net Effect (current + projected)
calculateNetEffect(baseline, meals, doses, ...): NetEffectResult
```

Used By:

- `useActiveEffects.ts` - Main consumer
- `useBloodGlucoseEstimation.ts` - Estimation logic
- `baseline.ts`, `net-effect.ts`, `cumulative-effects.ts` - Wrapper exports

Calls:

- `calculateStackedMealEffect()` from `meal-pharmacodynamics.ts`
- `calculateStackedInsulinEffect()` from `insulin/pharmacodynamics.ts`

- `getMealAbsorptionProfile()` from `meal-pharmacodynamics.ts`
- `getInsulinProfile()` from `insulin/pharmacodynamics.ts`
- `TimeManager` functions for timezone/reset logic

Backend Equivalent:

- `backend/utils/pharmacodynamics.py`
- Functions: `calculate_stable_baseline_from_reading()`, `calculate_total_cumulative_effects()`, `calculate_meal_cumulative_effect()`, `calculate_insulin_cumulative_effect()`

Debug Here If:

- Baseline values incorrect
- Cumulative effects wrong
- Net effect calculation off
- Daily reset not working
- Timezone issues

---

3. `utils/glucose/meal-pharmacodynamics.ts` ⭐⭐

Role: Meal absorption modeling using gamma distribution

Responsibilities:

- **Gamma Distribution** - Models carb absorption curves
- **Absorbed Fraction** - Calculates what % of carbs absorbed
- **Active MOB** - Current carbs still digesting (PRESENT→FUTURE)
- **Stacked Effect** - Combines multiple meals
- **BG Impact** - Converts carbs to mg/dL

Key Functions:

```typescript
```

```
// Gamma distribution (core math)
calculateGammaAbsorption(hoursSinceMeal, profile): number

// Meal activity at specific time
calculateMealActivity(meal, currentTime, constants): MealActivityResult

// Absorbed fraction (0.0 to 1.0)
calculateAbsorbedFraction(hoursSinceMeal, profile): number

// Multiple meals combined (for active MOB)
calculateStackedMealEffect(meals, currentTime, constants): StackedMealResult {
  totalMOB: number,          // Carbs still digesting
  totalBGElevation: number,  // Current rate of rise
  totalPendingRise: number,  // Future rise from MOB
  contributions: Array       // Per-meal breakdown
}
```

Used By:

- `blood-glucose-estimation.ts` - For cumulative and active effects
- `useActiveEffects.ts` - For MOB calculation

Backend Equivalent:

- `backend/utils/pharmacodynamics.py`
- Functions: `calculate_gamma_absorption()`, `calculate_meal_activity_percentage()`, `calculate_absorbed_fraction_trapezoidal()`

Debug Here If:

- MOB values incorrect
- Meal absorption too fast/slow
- Gamma distribution not working
- Carb impact calculations wrong

---

4. `utils/insulin/pharmacodynamics.ts` ⭐ ⭐

Role: Insulin activity modeling using gamma distribution

Responsibilities:

- **Insulin Activity Curves** - Based on S3 Guidelines
- **IOB Calculation** - Remaining active insulin

- **Absorbed Fraction** - What % of insulin has acted
- **Stacked Effect** - Combines multiple doses
- **BG Impact** - Converts insulin to mg/dL drop

Key Functions:

```typescript
// Insulin activity at specific time
calculateInsulinActivity(hoursSinceDose, dose, profile): InsulinActivityResult

// IOB calculation (remaining insulin)
calculateIOB(hoursSinceDose, dose, profile): number

// Multiple doses combined (for active IOB)
calculateStackedInsulinEffect(doses, correctionFactor): StackedInsulinEffect {
  totalIOB: number,          // Units still active
  totalBGImpact: number,     // Current rate of drop
  contributions: Array       // Per-dose breakdown
}
```

Used By:

- `blood-glucose-estimation.ts` - For cumulative and active effects
- `useActiveEffects.ts` - For IOB calculation

Backend Equivalent:

- `backend/utils/pharmacodynamics.py`
- Functions: `calculate_insulin_activity_percentage()`, `calculate_iob_trapezoidal()`

Debug Here If:

- IOB values incorrect
- Insulin acting too fast/slow
- Activity curves wrong
- Insulin impact calculations off

TIER 2: API LAYER (Backend Communication)

5. `services/api/insulin.ts` 💉

Role: Fetch insulin dose data from backend

Responsibilities:

- Get insulin doses for time period
- Get active insulin (IOB) from backend
- Log new insulin doses
- Get insulin analytics

Key Functions:

```typescript
getDoses(params): Promise<InsulinDataResponse>
getActiveInsulin(): Promise<ActiveInsulinResponse>
logDose(data): Promise<InsulinLogResponse>
```

Endpoints:

- GET `/api/insulin-data` - Historical doses
- GET `/api/insulin/active-effect` - Current IOB
- POST `/api/insulin/log` - Log new dose

Used By:

- `useActiveEffects.ts` - To fetch dose data
- Insulin logging screens

Debug Here If:

- No insulin doses showing
- IOB from backend is wrong
- Doses not saving

---

6. `services/api/meals.ts` 🍽️

Role: Fetch meal data from backend

Responsibilities:

- Get meals for time period
- Calculate meal insulin (backend calculation)
- Create new meals
- Get meal details

Key Functions:

```typescript
getMeals(params): Promise<MealsListResponse>
calculateMeal(data): Promise<MealCalculationResponse>
createMeal(data): Promise<MealResponse>
```

Endpoints:

- GET `/api/meals-only` - Historical meals
- POST `/api/meal/calculate` - Calculate insulin for meal
- POST `/api/meal` - Create new meal

Used By:

- `useActiveEffects.ts` - To fetch meal data
- Meal logging screens

Debug Here If:

- No meals showing
- Meal data incomplete
- Carb values wrong

---

7. `services/api/mob.ts` 📊

Role: Fetch meal on board data from backend

Responsibilities:

- Get current MOB from backend
- Get meal timing assessment
- Get list of active meals

Key Functions:

```typescript
getMealOnBoard(params): Promise<MealOnBoardResponse>
getMealTimingAssessment(): Promise<MealTimingAssessment>
getActiveMeals(): Promise<ActiveMealsResponse>
```

Endpoints:

- GET `/api/meal-on-board` - Current MOB

- GET `/api/meal-timing-assessment` - Safe to eat?
- GET `/api/active-meals` - Currently absorbing meals

Used By:

- Comparison component
- Meal timing warnings

Debug Here If:

- Backend MOB differs from frontend
- Active meals count wrong

---

8. `services/api/glucose.ts` 

Role: Fetch blood glucose readings from backend

Responsibilities:

- Get glucose readings for time period
- Create new readings
- Get latest reading

Key Functions:

```typescript
getReadings(params): Promise<BloodSugarResponse[]>
createReading(data): Promise<BloodSugarCreateResponse>
getLatestReading(): Promise<BloodSugarResponse | null>
```

Endpoints:

- GET `/api/blood-sugar` - Historical readings
- POST `/api/blood-sugar` - Log new reading

Used By:

- `useActiveEffects.ts` - To get recent reading for baseline
- Glucose logging screens

Debug Here If:

- No glucose readings showing
- Latest reading wrong

- Baseline calculation using old reading

---

9. `services/api/patient.ts`

Role: Fetch and update patient constants

Responsibilities:

- Get patient constants (ratios, factors, settings)
- Update patient constants
- Get patient profile

Key Functions:

```typescript
getConstants(): Promise<PatientConstantsResponse>
updateConstants(constants): Promise<PatientConstantsResponse>
```

Endpoints:

- GET `/api/patient/constants` - Get constants
- PUT `/api/patient/constants` - Update constants

Used By:

- `patient.store.ts` - State management
- `usePatientConstants.ts` - Hook wrapper
- Settings screens

Debug Here If:

- Constants not loading
- Wrong insulin ratios
- Missing timezone offset

TIER 3: STATE MANAGEMENT

10. `store/patient.store.ts`

Role: Zustand store for patient constants

Responsibilities:

- Load constants from API
- Cache constants (5min TTL)
- Update constants
- Provide fallback to defaults

Key Functions:

```typescript
loadConstants(): Promise<void>
updateConstants(constants): Promise<void>
getConstant(key): value
```

State:

```typescript
{
  constants: PatientConstants | null,
  activeConditions: string[],
  activeMedications: string[],
  medicationSchedules: object,
  isLoading: boolean,
  error: string | null
}
```

Used By:

- `usePatientConstants.ts` - Hook wrapper
- All calculation utilities (via constants parameter)

Debug Here If:

- Constants undefined
- Wrong default values
- Cache not working
- **TODO: updateConstants not saving to backend** ⚠️

TIER 4: TIME MANAGEMENT

11. `utils/time/TimeManager.ts` ⏰

Role: All time-related operations and daily reset logic

Responsibilities:

- **Timezone Conversion** - UTC ↔ Patient Local Time
- **Daily Reset** - Calculate when cumulative effects reset
- **Time Normalization** - Second/minute/hour boundaries
- **Duration Calculations** - Hours since meal/dose
- **Formatting** - Display-friendly time strings

Key Functions:

```typescript
// Daily reset (CRITICAL for cumulative effects)
getDailyResetTime(timestamp, resetHour, tzOffset): number
getLastResetTime(timestamp, resetHour, tzOffset): number

// Time calculations
calculateHoursSince(currentTime, eventTime): number
normalizeToSecondBoundary(timestamp): number

// Timezone
convertUTCToPatientLocal(utcTime, tzOffset): Date
convertPatientLocalToUTC(localTime, tzOffset): Date
```

Used By:

- `blood-glucose-estimation.ts` - For daily reset calculations
- All calculation utilities - For hours-since calculations

Backend Equivalent:

- `backend/time_manager.py`
- Functions: `get_daily_reset_time()`, `get_last_reset_time()`, `calculate_hours_since()`

Debug Here If:

- Daily reset at wrong time
- Timezone calculations wrong
- Hours-since calculations off
- Reset happening multiple times per day

TIER 5: CONSTANTS & CONFIGURATION

12. `constants/shared-constants.ts` 📋

Role: All application constants (auto-generated from backend)

Contains:

- **DEFAULT_PATIENT_CONSTANTS** - Default ratios, factors, settings
- **MEAL_ABSORPTION_PROFILES** - Gamma parameters for meal types
- **MEDICATION_FACTORS** - Insulin profiles from S3 Guidelines
- **ACTIVITY_LEVELS** - Activity coefficient mappings
- **Measurement conversions** - Volume/weight units

Critical Constants:

```typescript
DEFAULT_PATIENT_CONSTANTS = {
  insulin_to_carb_ratio: 10,
  correction_factor: 40,
  target_glucose: 100,
  carb_to_bg_factor: 4.0,
  daily_reset_hour: 7,
  timezone_offset_minutes: 0,  // ⚠️ Must be present
  // ... all other constants
}

MEAL_ABSORPTION_PROFILES = {
  very_fast: { onset_hours: 0.17, peak_hours: 1.0, duration_hours: 2.5, ... },
  fast: { onset_hours: 0.25, peak_hours: 1.5, duration_hours: 3.5, ... },
  medium: { onset_hours: 0.33, peak_hours: 2.0, duration_hours: 4.5, ... },
  slow: { onset_hours: 0.5, peak_hours: 2.5, duration_hours: 5.5, ... },
  very_slow: { onset_hours: 0.67, peak_hours: 3.0, duration_hours: 6.5, ... }
}
```

Generated From:

- `backend/constants.py` - Run `python constants.py` to regenerate

Used By:

- ALL calculation utilities
- `patient.store.ts` - For defaults
- `usePatientConstants.ts` - For fallback values

Debug Here If:

- Wrong default values
- Missing constants
- Absorption profiles incorrect
- **timezone_offset_minutes missing** ⚠️

TIER 6: TYPE DEFINITIONS

13. types/calculation.types.ts 🧮

Role: Type definitions for calculation results

Key Types:

```typescript
NetEffectResult          // Complete net effect with all data
BaselineResult           // Baseline calculation breakdown
BGEstimation             // Blood glucose estimation
CumulativeEffectsResult // Daily cumulative effects
TimelinePoint            // Point on BG timeline
```

14. types/pharmacodynamics.types.ts 💊

Role: Type definitions for PD/PK calculations

Key Types:

```typescript
PharmacodynamicProfile    // Absorption/activity profile
GammaDistributionParams   // Gamma curve parameters
ActivityCurvePoint        // Point on activity curve
IOBResult                 // Insulin on board result
MOBResult                 // Meal on board result
```

15. types/constants.types.ts 📋

Role: Type definitions for patient constants

Key Types:

```typescript
```

```
PatientConstants          // All patient-specific settings
MealAbsorptionProfile     // Meal absorption parameters
MedicationFactor          // Insulin pharmacokinetics
```

🔗 CALCULATION CHAIN

When User Opens Dashboard:

```
1. UI Component Renders
   └─> useActiveEffects() hook called

2. useActiveEffects() fetches data
   ├─> getDoses() → Backend: GET /api/insulin-data
   ├─> getMeals() → Backend: GET /api/meals-only
   ├─> getReadings() → Backend: GET /api/blood-sugar
   └─> usePatientConstants() → patient.store.ts → getConstants()

3. useActiveEffects() calculates baseline
   └─> calculateStableBaselineFromReading()
      ├─> Uses most recent glucose reading
      ├─> Loops through meals:
      │   ├─> getMealAbsorptionProfile()
      │   ├─> calculateAbsorbedFraction() [meal-pharmacodynamics.ts]
      │   └─> Calculates cumulative meal BG elevation
      ├─> Loops through doses:
      │   ├─> getInsulinProfile()
      │   ├─> calculateIOB() [insulin/pharmacodynamics.ts]
      │   └─> Calculates cumulative insulin BG reduction
      └─> Returns: stableBaseline = reading - (meal - insulin)

4. useActiveEffects() calculates current active effects
   ├─> calculateStackedMealEffect()
   │   ├─> For each meal:
   │   │   ├─> calculateMealActivity()
   │   │   ├─> calculateGammaAbsorption()
   │   │   └─> Calculate current BG elevation rate
   │   └─> Returns: { totalMOB, totalBGElevation, totalPendingRise }
   │
   └─> calculateStackedInsulinEffect()
      ├─> For each dose:
      │   ├─> calculateInsulinActivity()
      │   ├─> calculateIOB()
      │   └─> Calculate current BG reduction rate
      └─> Returns: { totalIOB, totalBGImpact }
```

5. useActiveEffects() calculates net effect
   └─> calculateNetEffect()
      ├─> Gets cumulative effects: calculateTotalCumulativeEffects()
      ├─> Combines baseline + cumulative + active
      ├─> Calculates: estimatedBG, projectedFinalBG
      ├─> Determines: safetyStatus, trend
      └─> Returns: Complete NetEffectResult

6. UI Component displays results
   └─> Shows: IOB, MOB, Baseline, Estimated BG, Projected BG, Trend

---

🔄 BACKEND-FRONTEND MAPPING

File Mapping:

Frontend File	Backend File	Purpose
`blood-glucose-estimation.ts`	`pharmacodynamics.py`	Core calculations
`meal-pharmacodynamics.ts`	`pharmacodynamics.py`	Meal absorption
`insulin/pharmacodynamics.ts`	`pharmacodynamics.py`	Insulin activity
`TimeManager.ts`	`time_manager.py`	Time/timezone logic
`shared-constants.ts`	`constants.py`	Constants & profiles
`useActiveEffects.ts`	N/A	Frontend-only orchestration
`services/api/insulin.ts`	`medication_routes.py`	IOB API
`services/api/meals.ts`	`meal_routes.py`	Meals API
`services/api/mob.ts`	`meal_routes.py`	MOB API
`services/api/patient.ts`	`patient_routes.py`	Constants API

Function Mapping:

Frontend Function	Backend Function	Location
`calculateStableBaselineFromReading()`	`calculate_stable_baseline_from_reading()`	blood-glucose-estimation.ts ↔ pharmacodynamics.py

Frontend Function	Backend Function	Location
`calculateTotalCumulativeEffects()`	`calculate_total_cumulative_effects()`	blood-glucose-estimation.ts ↔ pharmacodynamics.py
`calculateMealCumulativeEffect()`	`calculate_meal_cumulative_effect()`	blood-glucose-estimation.ts ↔ pharmacodynamics.py
`calculateInsulinCumulativeEffect()`	`calculate_insulin_cumulative_effect()`	blood-glucose-estimation.ts ↔ pharmacodynamics.py
`getDailyResetTime()`	`get_daily_reset_time()`	blood-glucose-estimation.ts ↔ time_manager.py
`calculateGammaAbsorption()`	`calculate_gamma_absorption()`	meal-pharmacodynamics.ts ↔ pharmacodynamics.py
`calculateAbsorbedFraction()`	`calculate_absorbed_fraction_trapezoidal()`	meal-pharmacodynamics.ts ↔ pharmacodynamics.py
`calculateIOB()`	`calculate_iob_trapezoidal()`	insulin/pharmacodynamics.ts ↔ pharmacodynamics.py
`calculateStackedMealEffect()`	N/A (calculated via cumulative)	meal-pharmacodynamics.ts
`calculateStackedInsulinEffect()`	N/A (calculated via cumulative)	insulin/pharmacodynamics.ts

🐛 DEBUGGING GUIDE

By Symptom:

Symptom: IOB value wrong

Check in order:

1. **API Response -** `services/api/insulin.ts`
   - Log `getActiveInsulin()` response
   - Verify doses are being fetched
2. **Insulin Activity -** `utils/insulin/pharmacodynamics.ts`
   - Check `calculateIOB()` function
   - Verify gamma distribution parameters
   - Check hours since dose calculation
3. **Cumulative Calculation -** `blood-glucose-estimation.ts`
   - Check `calculateInsulinCumulativeEffect()`

- Verify daily reset is working
- Check timezone handling

4. **Constants -** `shared-constants.ts`
    - Verify `correction_factor` is correct
    - Check insulin profiles in `medication_factors`

Symptom: MOB value wrong

Check in order:

1. **API Response -** `services/api/meals.ts`
    - Log `getMeals()` response
    - Verify meals are being fetched
    - Check carb values in meal data
2. **Meal Activity -** `utils/glucose/meal-pharmacodynamics.ts`
    - Check `calculateMealActivity()` function
    - Verify gamma distribution calculation
    - Check absorption profile lookup
3. **Cumulative Calculation -** `blood-glucose-estimation.ts`
    - Check `calculateMealCumulativeEffect()`
    - Verify carb equivalents calculation
    - Check daily reset timing
4. **Constants -** `shared-constants.ts`
    - Verify `carb_to_bg_factor` is correct
    - Check `MEAL_ABSORPTION_PROFILES`

Symptom: Baseline calculation wrong

Check in order:

1. **Reading Data -** `services/api/glucose.ts`
    - Log `getLatestReading()` response
    - Verify reading timestamp is correct
2. **Baseline Function -** `blood-glucose-estimation.ts`
    - Check `calculateStableBaselineFromReading()`

- Log cumulative meal effect
- Log cumulative insulin effect
- Verify formula: `baseline = reading - (meal - insulin)`

3. **Daily Reset -** `TimeManager.ts`
   - Check `getLastResetTime()` is correct
   - Verify timezone offset is applied
   - Check reset hour (default 7 AM)
4. **Meal/Insulin Loops -** `blood-glucose-estimation.ts`
   - Verify only meals AFTER reset are counted
   - Verify only doses AFTER reset are counted
   - Check absorbed fraction calculations

Symptom: Daily reset not working / happens at wrong time

Check in order:

1. **Timezone Offset -** `shared-constants.ts`
   - Verify `timezone_offset_minutes` is present
   - Check value is correct for patient's timezone
   - Example: UTC-5 (EST) = -300 minutes
2. **Reset Calculation -** `blood-glucose-estimation.ts`
   - Check `getDailyResetTime()` implementation
   - Log the calculated reset time
   - Verify it matches patient's local 7 AM
3. **TimeManager -** `utils/time/TimeManager.ts`
   - Check timezone conversion functions
   - Verify UTC ↔ Local conversion logic
4. **Patient Constants -** `store/patient.store.ts`
   - Verify constants loaded correctly
   - Check `daily_reset_hour` value
   - Check `timezone_offset_minutes` value

Symptom: Net effect calculation wrong

Check in order:

1. **Dependencies -** All of the above
    - Verify IOB is correct
    - Verify MOB is correct
    - Verify baseline is correct
2. **Net Effect Function -** `blood-glucose-estimation.ts`
    - Check `calculateNetEffect()` implementation
    - Log all intermediate values:

```typescript
console.log('Baseline:', baseline?.stableBaseline);
console.log('Cumulative Net:', cumulativeResult.cumulativeNetBaseline);
console.log('Active Meal Effect:', mealEffects.totalBGElevation);
console.log('Active Insulin Effect:', insulinEffects.totalBGImpact);
console.log('Pending MOB Rise:', mealEffects.totalPendingRise);
console.log('Pending IOB Drop:', insulinEffects.totalBGImpact);
```

3. **Formula Verification**

```typescript
estimatedBG = baseline + cumulativeNetBaseline

projectedFinalBG = baseline +
                   cumulativeNetBaseline +
                   pendingMOBRise -
                   pendingIOBDrop
```

4. **Safety Status -** `blood-glucose-estimation.ts`
    - Check `determineSafetyStatus()` thresholds
    - Check `determineTrend()` thresholds

Symptom: Values match backend sometimes but not others

Likely Causes:

1. **Timezone Issues -** Reset happening at different times
2. **Rounding Differences -** JavaScript vs Python floating point
3. **Race Conditions -** Frontend calculating before data fully loaded
4. **Stale Cache -** Using old constants or data

Solutions:

1. Add `timezone_offset_minutes` to constants
2. Use rounding utility (see IMMEDIATE_FIXES.md)
3. Add loading checks in `useActiveEffects.ts`
4. Clear cache or reduce TTL in `patient.store.ts`

By File:

If This File Has Issues	Symptoms	Debug Steps
`useActiveEffects.ts`	Nothing shows, errors, infinite loading	Check API calls, check data fetching, add console.logs
`blood-glucose-estimation.ts`	All calculations wrong	Check each function individually, compare to backend logs
`meal-pharmacodynamics.ts`	MOB wrong, meal absorption off	Check gamma distribution, verify absorption profiles
`insulin/pharmacodynamics.ts`	IOB wrong, insulin acting weird	Check insulin profiles, verify S3 Guidelines parameters
`TimeManager.ts`	Reset timing wrong	Check timezone calculations, verify reset hour
`shared-constants.ts`	Wrong defaults, missing constants	Regenerate from backend, check `timezone_offset_minutes`
`patient.store.ts`	Constants not loading	Check API call, check cache, check fallback to defaults
`services/api/insulin.ts`	No insulin data	Check endpoint, check backend logs, check authentication
`services/api/meals.ts`	No meal data	Check endpoint, check backend logs, check meal format
`services/api/patient.ts`	Constants undefined	Check backend route, check response format

---

📝 SUMMARY CHEAT SHEET

Quick Reference: "Which file do I check?"

IOB Issues:

1. `services/api/insulin.ts` - Data fetching
2. `utils/insulin/pharmacodynamics.ts` - Calculation
3. `blood-glucose-estimation.ts` - Integration

MOB Issues:

1. `services/api/meals.ts` - Data fetching
2. `utils/glucose/meal-pharmacodynamics.ts` - Calculation
3. `blood-glucose-estimation.ts` - Integration

Baseline Issues:

1. `blood-glucose-estimation.ts` - `calculateStableBaselineFromReading()`
2. `TimeManager.ts` - Daily reset logic
3. `shared-constants.ts` - Timezone offset

Net Effect Issues:

1. `blood-glucose-estimation.ts` - `calculateNetEffect()`
2. Check all of the above (IOB, MOB, Baseline)

Timezone Issues:

1. `shared-constants.ts` - `timezone_offset_minutes` present?
2. `TimeManager.ts` - Conversion logic
3. `blood-glucose-estimation.ts` - Reset time calculation

Constants Issues:

1. `shared-constants.ts` - Values correct?
2. `backend/constants.py` - Regenerate if needed
3. `store/patient.store.ts` - Loading logic

Priority Order for Debugging:

1. **Check constants loaded** - `usePatientConstants.ts`
2. **Check data fetched** - `useActiveEffects.ts`
3. **Check core calculation** - `blood-glucose-estimation.ts`

4. **Check component calculations -** `meal-pharmacodynamics.ts`, `insulin/pharmacodynamics.ts`
5. **Check time logic -** `TimeManager.ts`
6. **Check API communication -** `services/api/*`

🎓 LEARNING PATH

To Understand Active Effects:

Beginner:

1. Read `useActiveEffects.ts` - See the big picture
2. Read type definitions in `types/calculation.types.ts`
3. Read `shared-constants.ts` - Understand default values

Intermediate: 4. Read `blood-glucose-estimation.ts` - Core logic 5. Read `TimeManager.ts` - Time/timezone handling 6. Compare frontend formulas to backend (FORMULA_COMPARISON.md)

Advanced: 7. Read `meal-pharmacodynamics.ts` - Gamma distribution 8. Read `insulin/pharmacodynamics.ts` - S3 Guidelines 9. Trace a complete calculation end-to-end with debugger

---

END OF DOCUMENT

This document maps every file involved in Active Effects calculations and explains what each one does. Use it as your debugging roadmap!